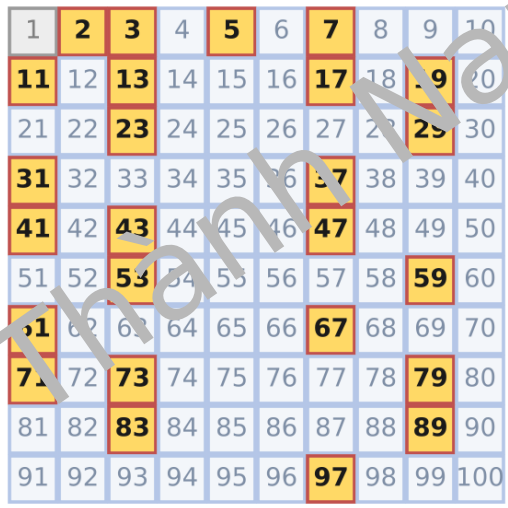
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

SỐ HỌC

Từ cơ bản đến nâng cao

TẬP 1 · LÝ THUYẾT



Lê Thành Nam

Chương 1. CHIA HẾT VÀ ƯỚC SỐ
Chương 2. SỐ NGUYÊN TỐ
Chương 3. SỐ HỌC ĐỒNG DƯ
Chương 4. TỔ HỢP VÀ ĐẾM
Chương 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH

LỜI NÓI ĐẦU

Khoảng năm 300 trước Công nguyên, trong bộ sách *Cơ sở*, Euclid mô tả một cách tìm ước chung lớn nhất của hai số chỉ bằng những phép trừ liên tiếp. Hơn hai nghìn năm sau, chính thuật toán ấy chạy hàng tỉ lần mỗi ngày bên trong các máy tính: trong phần mềm mã hóa bảo vệ tài khoản ngân hàng, trong các thư viện tính toán, và trong gần như mọi đề thi học sinh giỏi Tin học có một chút "mùi số học".

Học sinh thường gặp ba khó khăn với các bài số học. Thứ nhất, **số quá lớn**: đề cho n tới 10^{12}, 10^{18}, có khi là một số dài cả triệu chữ số, và mọi vòng lặp "thử từng số" đều chạy không kịp. Thứ hai, **những cái bẫy âm thầm**: tràn số, số dư âm, chia trong đồng dư..., chương trình không báo lỗi mà chỉ in ra kết quả sai. Thứ ba, các công thức thường được học thuộc như những "câu thần chú", nên khi đề bài thay đổi một chút là không biết bắt đầu từ đâu.

Cuốn sách này được viết để giải quyết cả ba khó khăn ấy. Mỗi bài học không đưa ra công thức ngay. Em sẽ được đặt trước một bài toán cụ thể, viết chương trình ngây thơ (có sơ đồ khối và code đầy đủ), đếm số phép tính hoặc chỉ ra chỗ tràn số để thấy bế tắc, rồi được dẫn từng bước tới ý tưởng tốt, kèm chứng minh vì sao nó đúng. Mọi ví dụ đều được chạy tay bằng bảng và hình vẽ; các con số trong bảng được sinh ra bằng cách chạy chính thuật toán, nên em có thể tin từng con số.

Bộ tài liệu gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 20 bài học trong 5 chương: Chia hết và ước số; Số nguyên tố; Số học đồng dư; Tổ hợp và đếm; Kĩ thuật số học trong lập trình.
- **Tập 2: Bài tập** gồm các bài tập theo từng bài học, ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc theo subtask như đề thi thật. Tên các bài tập được giới thiệu ở mục *Luyện tập* cuối mỗi bài học.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm theo thuật toán.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với chương trình ngây thơ (hoặc với phép tính số lớn chính xác) trên hàng trăm bộ dữ liệu ngẫu nhiên.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu chuyện, một câu đố gần gũi để em bắt đầu nghĩ.
Kiến thức nền	Định nghĩa, định lí, hình vẽ trực quan và ví dụ tính tay.
Bài toán	Đề bài cụ thể, có ví dụ được sinh bằng chính chương trình trong sách.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm sơ đồ khối, chương trình đầy đủ và phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm (hoặc sai, hoặc tràn số), và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: ý tưởng, chạy tay bằng hình và bảng, chứng minh, chương trình.
Mở rộng	Các biến thể và ứng dụng thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Giới thiệu các bài tập tương ứng ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
ĐỊNH NGHĨA	Khái niệm mới cần nắm chính xác.
ĐỊNH LÍ	Kết quả quan trọng, thường kèm chứng minh.
THUẬT TOÁN	Các bước của thuật toán, viết bằng lời.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp: $O(\sqrt{n})$, $O(\log n)$, $O(n \log \log n)$...
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
HÃY HÌNH DUNG	Hình ảnh trực quan giúp nhớ lâu.
VÍ DỤ	Một ví dụ tính tay ngắn.
LƯU Ý	Chi tiết nhỏ nhưng dễ sai.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Lê Thành Nam

Quy ước màu trong hình vẽ

Màu	Ý nghĩa
Ô vàng viền đỏ	Giá trị đang xét ở bước đó, hoặc kết quả cuối cùng.
Ô xanh dương	Giá trị đã tính xong.
Ô cam nhạt	Dữ liệu vào (chữ số, số hạng...).
Ô xám	Giá trị khởi tạo, hoặc số đã bị loại (ví dụ: hợp số trong sàng).
Chữ đỏ	Kết luận, đáp số, hoặc chỗ cần chú ý.

Kí hiệu dùng trong sách

Kí hiệu	Ý nghĩa
$a \bmod m$	Số dư (không âm) của a khi chia cho m.
$\lfloor x \rfloor$, $\lceil x \rceil$	Phần nguyên dưới (số nguyên lớn nhất $\le x$), phần nguyên trên (số nguyên nhỏ nhất $\ge x$).
$b \mid a$	b chia hết a (a chia hết cho b).
$a \equiv b \pmod m$	a và b đồng dư theo mô-đun m: $m \mid (a - b)$.
gcd(a, b), lcm(a, b)	Ước chung lớn nhất, bội chung nhỏ nhất (trong lời văn viết ƯCLN, BCNN).
p	Thường dùng cho số nguyên tố.
d(n), σ(n), φ(n)	Số ước, tổng các ước, số các số trong 1..n nguyên tố cùng nhau với n.
C(n, k)	Số cách chọn k phần tử từ n phần tử (tổ hợp chập k của n).
O(...)	Độ phức tạp thời gian. Máy tính làm khoảng 10^8 phép tính đơn giản mỗi giây.

Lê Thành Nam

LỜI KHUYÊN KHI HỌC

Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một bảng chạy tay, hãy che bảng lại và tự tính trước. Mỗi khi gặp một định lí, hãy thử nó với vài số nhỏ trước khi đọc chứng minh. Và mỗi khi viết một chương trình số học, hãy tự hỏi ba câu: **giá trị lớn nhất có thể là bao nhiêu? có số âm không? có phép chia nào không?** Ba câu hỏi ấy chặn được phần lớn các lỗi số học.

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. CHIA HẾT VÀ ƯỚC SỐ **6**

Bài 1. Phép chia có dư và đồng dư 7
Bài 2. Ước số: liệt kê, đếm và tính tổng 16
Bài 3. Ước chung lớn nhất và bội chung nhỏ nhất 23
Bài 4. Euclid mở rộng và phương trình ax + by = c 31

CHƯƠNG 2. SỐ NGUYÊN TỐ **39**

Bài 5. Số nguyên tố và kiểm tra nguyên tố 40
Bài 6. Sàng Eratosthenes 53
Bài 7. Phân tích thừa số nguyên tố 65
Bài 8. Sàng trên đoạn 75

CHƯƠNG 3. SỐ HỌC ĐỒNG DƯ **85**

Bài 9. Lũy thừa nhanh 86
Bài 10. Nghịch đảo mô-đun 98
Bài 11. Hàm phi Euler và định lí Euler 110
Bài 12. Định lí số dư Trung Hoa 122

CHƯƠNG 4. TỔ HỢP VÀ ĐẾM **133**

Bài 13. Tổ hợp và hệ số nhị thức 134
Bài 14. Nguyên lí bù trừ 148
Bài 15. Dãy truy hồi và lũy thừa ma trận 160
Bài 16. Kĩ thuật chia khối [n / i] 172

CHƯƠNG 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH **182**

Bài 17. Số nguyên lớn 183
Bài 18. Hệ cơ số và biểu diễn nhị phân 196
Bài 19. Tổng tiền tố và số dư 207
Bài 20. Điểm nguyên và hình học số nguyên 217

PHỤ LỤC A. TỔNG HỢP THUẬT TOÁN VÀ CÔNG THỨC **227**

PHỤ LỤC B. NHỮNG CON SỐ ĐÁNG NHỚ **230**

Lê Thành Nam

CHƯƠNG 1. CHIA HẾT VÀ ƯỚC SỐ

Mọi thứ trong số học bắt đầu từ một phép tính em đã biết từ năm lớp 3: phép chia có dư. Từ đó nảy ra những câu hỏi rất "Tin học": số dư của một số dài một triệu chữ số là bao nhiêu? Một số có bao nhiêu ước? Làm sao tìm ước chung lớn nhất của hai số cỡ 10^{18} trong nháy mắt? Phương trình $3x + 5y = 2026$ có nghiệm nguyên không?

Chương này xây nền móng cho cả cuốn sách: đồng dư, ước số, ước chung lớn nhất và thuật toán Euclid. Thuật toán Euclid ra đời hơn 2300 năm trước, nhưng đến nay vẫn là một trong những thuật toán được dùng nhiều nhất trong lập trình thi đấu.

TRONG CHƯƠNG NÀY

Bài 1. Phép chia có dư và đồng dư: thương, số dư và số dư của một số rất lớn

Bài 2. Ước số: liệt kê, đếm và tính tổng: các ước đi theo cặp, chỉ cần duyệt tới $\sqrt{n}$

Bài 3. Ước chung lớn nhất và bội chung nhỏ nhất: thuật toán Euclid

Bài 4. Euclid mở rộng và phương trình $ax + by = c$: định lí Bézout và họ nghiệm

Lê Thành Nam

BÀI 1 Phép chia có dư và đồng dư

Học xong bài này, em sẽ:

- ✓ Hiểu chính xác **thương** và **số dư** của phép chia, kể cả khi số bị chia là số âm.
- ✓ Biết phép `/` và `%` của C++ khác quy ước toán học ở chỗ nào, và cách sửa trong một dòng.
- ✓ Nắm khái niệm **đồng dư** $a \equiv b \pmod m$ và các tính chất cộng, trừ, nhân của nó.
- ✓ Tính được số dư của một số có tới hàng triệu chữ số trong O(độ dài), không cần lưu cả số.

KHỞI ĐỘNG

Hôm nay là **thứ Hai**. Cô giáo hỏi: "1000 ngày nữa là thứ mấy? Còn 100 ngày trước là thứ mấy?" Cả lớp đếm trên lịch một lúc rồi bỏ cuộc. Một bạn ngồi cuối lớp trả lời sau mười giây: "Chủ nhật, và thứ Bảy ạ."

Bạn ấy không đếm từng ngày. Bạn ấy chỉ cần biết 1000 **chia cho 7 dư bao nhiêu**. Bài học này cho em cách nghĩ ấy, rồi dùng nó để tính số dư của một số dài một triệu chữ số, thứ mà không kiểu số nào của C++ chứa nổi.

1. Phép chia có dư

Chia 17 cái kẹo cho 5 bạn, mỗi bạn được 3 cái và còn thừa 2 cái. Ta viết $17 = 5 \cdot 3 + 2$: **3 là thương**, **2 là số dư**. Số dư phải nhỏ hơn số bạn: nếu còn thừa từ 5 cái trở lên thì chia thêm được một lượt nữa.

ĐỊNH NGHĨA (PHÉP CHIA CÓ DƯ)

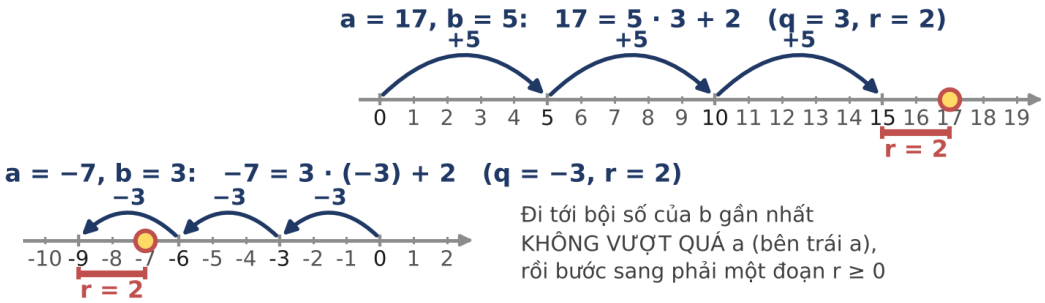
Với số nguyên a và số nguyên dương b, luôn tồn tại **duy nhất** một cặp số nguyên q, r sao cho:

$$a = b \cdot q + r \quad \text{và} \quad 0 \le r < b$$

q gọi là **thương**, r gọi là **số dư** của phép chia a cho b. Kí hiệu: $q = \lfloor a / b \rfloor$ (phần nguyên dưới, tức số nguyên lớn nhất không vượt quá a / b), $r = a \bmod b$. Nếu $r = 0$ ta nói a **chia hết** cho b, viết $b \mid a$.

Trên trục số, phép chia có dư có một hình ảnh rất dễ nhớ: đi từ 0 theo những bước nhảy dài b, dừng ở **bội số của b gần a nhất mà không vượt quá a**. Số bước nhảy là q, đoạn còn lại tới a là r. Cách nhìn này đúng cả khi a âm: khi đó ta nhảy sang trái.

Lê Thành Nam



Hình 1.1. *Thương là số bước nhảy, số dư là đoạn còn lại; số dư luôn thuộc [0, b).*

VÌ SAO CẶP (q, r) LUÔN TỒN TẠI VÀ DUY NHẤT?

Tồn tại: các bội số của b chia trục số thành những đoạn dài b liền nhau. Số a rơi vào đúng một đoạn $[b \cdot q, b \cdot q + b)$, nên $r = a - b \cdot q$ thỏa $0 \le r < b$.

Duy nhất: giả sử $a = b \cdot q + r = b \cdot q' + r'$ với $0 \le r, r' < b$. Trừ vế: $b \cdot (q - q') = r' - r$. Vế phải nằm trong khoảng $(-b, b)$ mà lại là bội của b, nên nó bằng 0. Vậy $r = r'$, $q = q'$.

2. Phép chia trong C++ và số âm

Với $a \ge 0$, phép `a / b` và `a % b` của C++ cho đúng q và r. Nhưng với a âm, C++ **làm tròn thương về phía 0** chứ không làm tròn xuống, nên số dư có thể âm:

a	b	a / b (C++)	a % b (C++)	q (toán học)	r (toán học)
17	5	3	2	3	2
−17	5	**−3**	**−2**	−4	3
−7	3	**−2**	**−1**	−3	2
−6	3	−2	0	−2	0

Quy tắc của C++ (từ chuẩn C++11): `a / b` bỏ phần thập phân (làm tròn về 0), còn `a % b` mang **dấu của a** và luôn thỏa `a == (a / b) * b + a % b`. Khi đề bài cần số dư theo nghĩa toán học (luôn không âm), hãy dùng công thức:

$$\mathbf{r = ((a \ \%\ m) + m) \ \%\ m}$$

Nếu a % m đã không âm thì cộng m rồi lấy dư lần nữa không làm thay đổi gì; nếu a % m âm (thuộc khoảng $(-m, 0)$) thì cộng m đưa nó về đúng khoảng $[0, m)$.

Lê Thành Nam

VÍ DỤ: THỨ TRONG TUẦN

Đánh số thứ Hai = 0, thứ Ba = 1, ..., Chủ nhật = 6. Sau (hoặc trước) k ngày kể từ ngày t thì thứ là (t + k) mod 7, với k âm nghĩa là lùi về quá khứ.

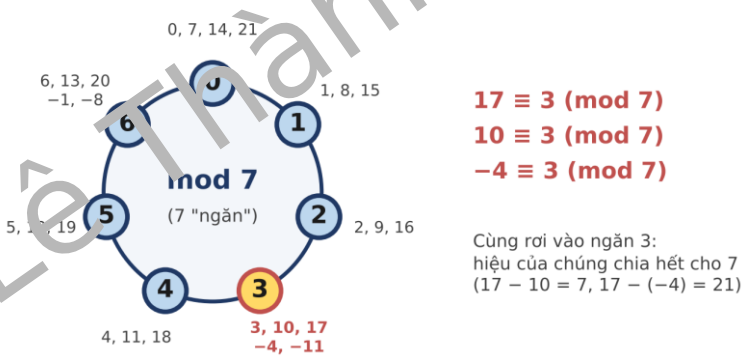
- 1000 ngày nữa: $1000 = 7 \cdot 142 + 6$, ra 6: **Chủ nhật**.
- 100 ngày trước: $-100 = 7 \cdot (-15) + 5$, ra 5: **thứ Bảy**. Trong C++, `-100 % 7` bằng -2: phải sửa thành `((-100 % 7) + 7) % 7 = 5`.
- 10^{18} ngày nữa: $10^{18} \bmod 7 = 1$: **thứ Ba**. (Em sẽ tự tính được số này ở cuối bài.)

3. Đồng dư

Trên "đồng hồ 7 ngăn" ở Hình 1.2, các số 3, 10, 17, −4 đều rơi vào cùng ngăn 3. Chúng khác nhau, nhưng với câu hỏi "thứ mấy?" thì chúng **như nhau**. Toán học có một từ riêng cho chữ "như nhau" ấy.

ĐỊNH NGHĨA (ĐỒNG DƯ)

Cho số nguyên dương m. Hai số nguyên a, b gọi là **đồng dư theo mô-đun m**, viết $a \equiv b \pmod m$, nếu chúng có cùng số dư khi chia cho m; tương đương: **m chia hết a − b**.



Hình 1.2. Mỗi số nguyên rơi vào đúng một trong m "ngăn" 0, 1, ..., m − 1. Các số cùng ngăn thì đồng dư với nhau.

Sức mạnh của đồng dư nằm ở chỗ: khi chỉ quan tâm đến số dư, ta được phép **thay một số bằng số dư của nó** ở bất kì đâu trong một biểu thức chỉ có cộng, trừ, nhân.

Lê Thành Nam

ĐỊNH LÍ (TÍNH CHẤT CỦA ĐỒNG DƯ)

Nếu $a \equiv a' \pmod m$ và $b \equiv b' \pmod m$ thì:

- $a + b \equiv a' + b' \pmod m$
- $a - b \equiv a' - b' \pmod m$
- $a \cdot b \equiv a' \cdot b' \pmod m$
- $a^k \equiv a'^k \pmod m$ với mọi số tự nhiên k.

Dạng hay dùng trong lập trình: $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$, và $(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$.

VÌ SAO ĐÚNG?

Đặt $a = a' + m \cdot s$ và $b = b' + m \cdot t$. Khi đó $a + b = (a' + b') + m \cdot (s + t)$, nên $a + b$ và $a' + b'$ chênh nhau một bội của m. Với phép nhân: $a \cdot b = a' \cdot b' + m \cdot (a' \cdot t + b' \cdot s + m \cdot s \cdot t)$: lại chênh nhau một bội của m. Lũy thừa là nhân nhiều lần.

CẨN THẬN VỚI PHÉP CHIA

Đồng dư **không** cho phép chia hai vế một cách tùy tiện: $2 \cdot 3 \equiv 2 \cdot 8 \pmod{10}$ (vì 6 và 16 cùng dư 6), nhưng $3 \not\equiv 8 \pmod{10}$. Phép "chia" trong đồng dư cần một công cụ riêng là **nghịch đảo mô-đun**, em sẽ học ở Bài 10.

Lê Thành Nam

4. Bài toán SODU

BÀI TOÁN: SODU

Cho một số nguyên không âm N rất lớn, có tới 10^6 chữ số, và số nguyên dương m. Hãy tính số dư của N khi chia cho m.

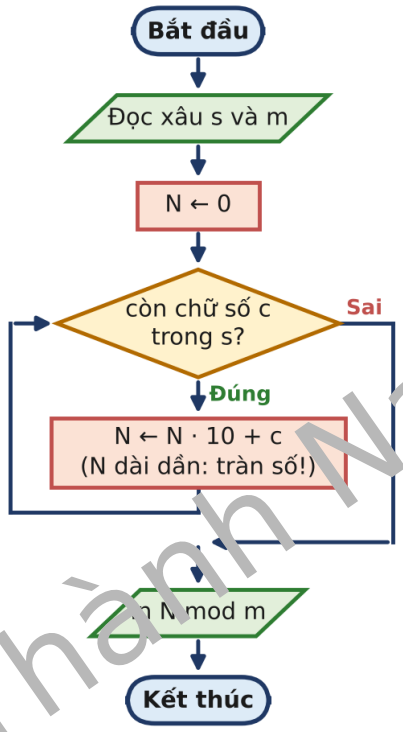
Dữ liệu vào: Dòng đầu chứa N (không có chữ số 0 thừa ở đầu). Dòng thứ hai chứa m ($1 \le m \le 10^9$).

Kết quả: Một số nguyên là N mod m.

SODU.INP	SODU.OUT	Giải thích
2026 7	3	$2026 = 7 \cdot 289 + 3$.
123456789012345678901234567890 1000000007	197434842	N có 30 chữ số, vượt xa giới hạn của mọi kiểu số nguyên trong C++.
7 9	7	Số bị chia nhỏ hơn số chia: thương bằng 0, số dư chính là N.

5. Ý tưởng ngây thơ: Dựng lại cả số N

Cách tự nhiên nhất: đọc N, đổi ra một số nguyên, rồi lấy dư. Vì N được cho dưới dạng các chữ số nên ta đọc vào xâu và dựng lại N từ trái sang phải: mỗi chữ số mới làm N gấp 10 lần rồi cộng thêm chữ số đó.



Hình 1.3. *Sơ đồ khối của cách dựng lại cả số N.*

Lê Thành Nam

SODU.CPP (ngây thơ): dựng lại N bằng unsigned long long

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("SODU.INP", "r")) {
        freopen("SODU.INP", "r", stdin);
        freopen("SODU.OUT", "w", stdout);
    }
    string s;
    long long m;
    cin >> s >> m;
    unsigned long long N = 0;    // dựng lại cả số N rồi mới chia
    for (char c : s) N = N * 10 + (c - '0');  // tràn khi N > 19 chữ số!
    cout << N % m << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Chương trình chỉ có một vòng lặp qua các chữ số: O(L) phép tính với L là độ dài của N, rất nhanh. Vấn đề không nằm ở thời gian mà ở **độ lớn**: `unsigned long long` chỉ chứa được số nhỏ hơn $2^{64} \approx 1{,}8 \cdot 10^{19}$, tức khoảng 19 chữ số.

BẾ TẮC

Với ví dụ 2 (30 chữ số), N bị **tràn số** từ chữ số thứ 20: máy tính âm thầm bỏ đi phần vượt quá 2^{64} và chương trình in ra một con số sai mà không báo lỗi gì. Một số có 10^6 chữ số cần khoảng 3,3 triệu bit, không kiểu số nào của C++ chứa nổi. Nhưng ta có thật sự cần **cả** số N không? Đề bài chỉ hỏi số dư của nó.

6. Phá bế tắc: Chỉ giữ lại số dư

Bước 1 Viết N theo sơ đồ Horner

Cách dựng N từ trái sang phải chính là cách viết N theo **sơ đồ Horner**. Ví dụ với N = 2026:

$$2026 = ((2 \cdot 10 + 0) \cdot 10 + 2) \cdot 10 + 6$$

Biểu thức này chỉ gồm phép nhân và phép cộng. Theo định lí về tính chất đồng dư, ở **mỗi bước** ta được thay kết quả trung gian bằng số dư của nó khi chia cho m, mà số dư cuối cùng không đổi.

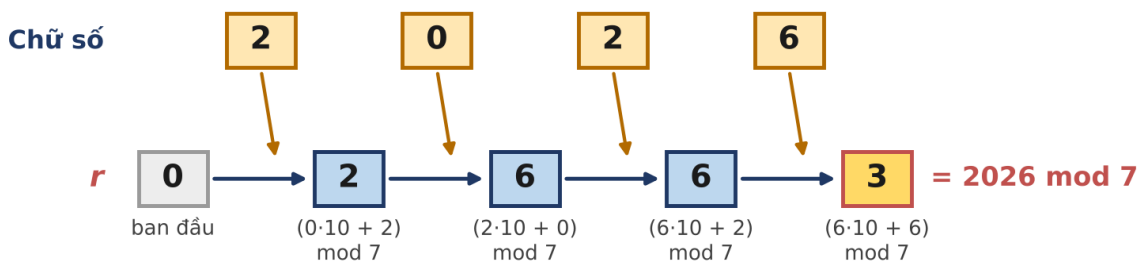
Bước 2 Duy trì số dư của phần đầu

Gọi r là số dư của phần đầu của N (các chữ số đã đọc) khi chia cho m. Ban đầu chưa đọc chữ số nào: r = 0. Đọc thêm chữ số c thì phần đầu mới bằng (phần đầu cũ) · 10 + c, nên:

$$r \leftarrow (r \cdot 10 + c) \bmod m$$

Vì luôn có $0 \le r < m$ nên $r \cdot 10 + c < 10 \cdot m \le 10^{10}$: vừa vặn trong `long long`, không bao giờ tràn.

Bước 3 Chạy tay với N = 2026, m = 7



Hình 1.4. Mỗi chữ số mới đẩy r sang một giá trị mới; r luôn nhỏ hơn 7.

Lê Thanh Nam

Chữ số đọc	Phần đầu của N	r · 10 + c	r mới (mod 7)
2	2	0 · 10 + 2 = 2	2
0	20	2 · 10 + 0 = 20	20 mod 7 = **6**
2	202	6 · 10 + 2 = 62	62 mod 7 = **6**
6	2026	6 · 10 + 6 = 66	66 mod 7 = **3**

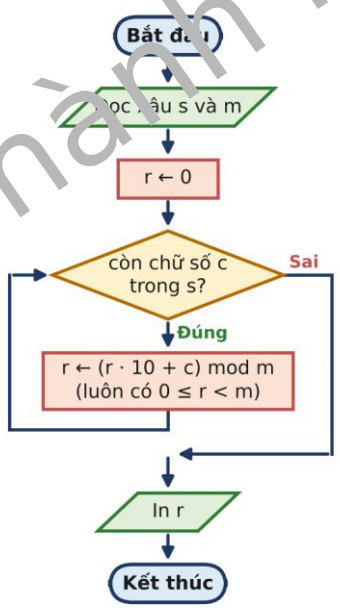
Kiểm tra lại cột thứ hai: 20 mod 7 = 6, 202 = 7 · 28 + 6, 2026 = 7 · 289 + 3. Đúng như bảng: r luôn là số dư của phần đầu, dù ta không hề lưu phần đầu ấy.

VÌ SAO ĐÚNG?

Bất biến: sau khi đọc k chữ số đầu, r là số dư của số tạo bởi k chữ số đó khi chia cho m.

Đúng khi k = 0 (r = 0). Giả sử đúng tới k, gọi phần đầu là A, ta có A ≡ r (mod m). Phần đầu mới là A · 10 + c ≡ r · 10 + c (mod m) theo tính chất đồng dư, nên (r · 10 + c) mod m chính là số dư của phần đầu mới. Khi k = L, phần đầu chính là N.

Bước 4 Chương trình



Hình 1.5. Sơ đồ khối của cách chỉ giữ số dư.

Lê Thành Nam

SODU.CPP: sơ đồ Horner, chỉ giữ số dư

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    if (fopen("SODU.INP", "r")) {
        freopen("SODU.INP", "r", stdin);
        freopen("SODU.OUT", "w", stdout);
    }
    string s;                // N dài tới 10^6 chữ số: đọc vào xâu
    long long m;
    cin >> s >> m;
    long long r = 0;         // r = (phần đầu đã đọc của N) mod m
    for (char c : s)
        r = (r * 10 + (c - '0')) % m;   // thêm một chữ số bên phải
    cout << r << "\n";
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Mỗi chữ số tốn một phép nhân, một phép cộng, một phép lấy dư: O(L) thời gian và O(1) bộ nhớ ngoài xâu đầu vào. Với L = 10^6, chương trình chạy chưa tới một phần trăm giây.

7. Mở rộng

Mở rộng 1: Dấu hiệu chia hết cho 3, 9 và 11

Vì $10 \equiv 1 \pmod 9$, mọi lũy thừa $10^k \equiv 1 \pmod 9$. Do đó một số đồng dư với **tổng các chữ số** của nó theo mô-đun 9 (và theo mô-đun 3). Đây là lí do của dấu hiệu chia hết cho 3 và 9 em đã học ở tiểu học. Tương tự, $10 \equiv -1 \pmod{11}$, nên $10^k \equiv (-1)^k \pmod{11}$: một số đồng dư với **tổng đan dấu** các chữ số của nó tính từ hàng đơn vị. Ví dụ 918 082: $2 - 8 + 0 - 8 + 1 - 9 = -22$, chia hết cho 11, nên 918 082 chia hết cho 11.

Mở rộng 2: Số dư của tổng và tích rất dài

Khi cần $(a_1 + a_2 + \ldots + a_n) \bmod m$ hay $(a_1 \cdot a_2 \cdot \ldots \cdot a_n) \bmod m$, hãy lấy dư **sau mỗi phép tính**, đúng như trong SODU. Với tích, nhớ rằng hai số nhỏ hơn $m \le 10^9$ nhân nhau được số tới 10^{18}: vẫn vừa `long long`. Khi m tới 10^{18}, tích hai số dư có thể tới 10^{36}: phải dùng `__int128` hoặc phép nhân đặc biệt ở Bài 9.

Mở rộng 3: Chữ số tận cùng và tính tuần hoàn

Chữ số tận cùng của a là a mod 10. Các lũy thừa $2^1, 2^2, 2^3, \ldots$ có chữ số tận cùng 2, 4, 8, 6, 2, 4, 8, 6, ...: lặp lại với **chu kì** 4, vì chỉ có hữu hạn số dư nên dãy số dư sớm muộn phải lặp. Ví dụ $2026 \bmod 4 = 2$, nên 2^{2026} có chữ số tận cùng giống 2^2, tức là **4**.

Lê Thành Nam

BẪY THƯỜNG GẶP

- Số dư âm: trong C++, -7 % 3 bằng −1, không phải 2. Dùng ((a % m) + m) % m khi a có thể âm.
- Tràn số trung gian: dùng int cho r khi m = 109 thì r · 10 + c có thể tới khoảng 1010 > 231. Hãy dùng long long.
- Lấy dư quá muộn: cộng dồn cả dãy rồi mới lấy dư thì tổng đã tràn từ trước. Lấy dư ngay sau mỗi phép tính.
- Chia hai vế của đồng dư: $2 \cdot 3 \equiv 2 \cdot 8 \pmod{10}$ nhưng $3 \not\equiv 8 \pmod{10}$. Đừng "giản ước" khi chưa học Bài 10.
- Đọc N vào kiểu số: cin >> N với N 30 chữ số không báo lỗi rõ ràng, chỉ cho kết quả sai. Đọc vào xâu.

GHI NHỚ

- $a = b \cdot q + r$ với $0 \le r < b$; $q = \lfloor a / b \rfloor$ là bội số lớn nhất của b không vượt quá a (tính theo số lần b).
- C++ làm tròn thương về 0; số dư theo nghĩa toán học là `((a % m) + m) % m`.
- $a \equiv b \pmod{m}$ khi và chỉ khi $m \mid (a - b)$. Đồng dư giữ nguyên qua cộng, trừ, nhân, lũy thừa; không giữ qua phép chia.
- Sơ đồ Horner: $r \leftarrow (r \cdot 10 + c) \bmod m$ cho số dư của số rất lớn trong O(độ dài).

Luyện tập

Bài	Tên	Gợi ý liên hệ
1.1	NGÀY TRONG TUẦN	Thứ của ngày cách hôm nay k ngày, $\lvert k\rvert \le 10^{18}$, k có thể âm. Số dư không âm.
1.2	CHỮ SỐ CUỐI	Chữ số tận cùng của a^b với a, b cho dưới dạng xâu rất dài: dùng tính tuần hoàn (chu kì 4) và số dư của số lớn.
1.3	VIẾT LIỀN	Số dư khi chia cho m của số 123456789101112...n (viết liền 1..n): sơ đồ Horner theo từng số, nhân với $10^{(\text{số chữ số})}$.

Tự kiểm tra nhanh

1) Theo quy ước toán học, (−23) mod 5 bằng bao nhiêu? Trong C++, `-23 % 5` cho kết quả gì? *(Đáp số: 2 và −3)*

2) Hôm nay là thứ Tư. Sau 2026 ngày là thứ mấy? *(Đáp số: 2026 mod 7 = 3, nên là thứ Bảy)*

3) Không dùng máy tính, tìm số dư của 123 456 789 khi chia cho 9 và khi chia cho 11. *(Đáp số: tổng chữ số 45 nên dư 0 khi chia cho 9; tổng đan dấu 9 − 8 + 7 − 6 + 5 − 4 + 3 − 2 + 1 = 5 nên dư 5 khi chia cho 11)*

Lê Thành Nam